

W6D4 – Esercitazione Python

Studente: Riccardo Luzzi
Corso: Cyber Security
Sistema utilizzato: Kali Linux
Ambiente di sviluppo: Visual Studio Code
Linguaggio: Python3

Introduzione

In questa esercitazione ho utilizzato Visual Studio Code su Kali Linux per realizzare ed eseguire due programmi in Python.

Nel primo esercizio ho creato una funzione che riceve una lista di parole e restituisce una seconda lista contenente la lunghezza di ogni parola.

Nel secondo esercizio, facoltativo, ho realizzato un generatore di password che permette all'utente di scegliere tra una password semplice di 8 caratteri e una password più complessa di 20 caratteri.

1. Preparazione dell'ambiente di lavoro

Per svolgere questa esercitazione ho utilizzato Visual Studio Code su Kali Linux.

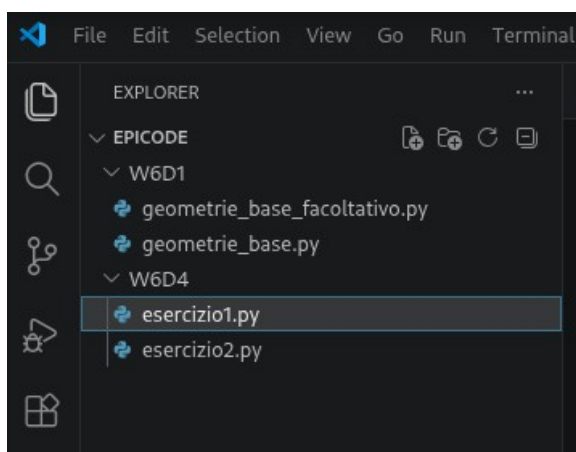
Ho aperto la cartella Epicode all'interno di Visual Studio Code e, per mantenere ordinati i file, ho utilizzato la directory: "W6D4".

All'interno della cartella ho creato due file Python:

- esercizio1.py
- esercizio2.py

Il primo file contiene l'esercizio sulla lunghezza delle parole, mentre il secondo contiene l'esercizio facoltativo relativo alla generazione delle password.

Figura 1 – Cartella W6D4 e file Python aperti in Visual Studio Code



2. Esercizio base

2.1 Creazione del file

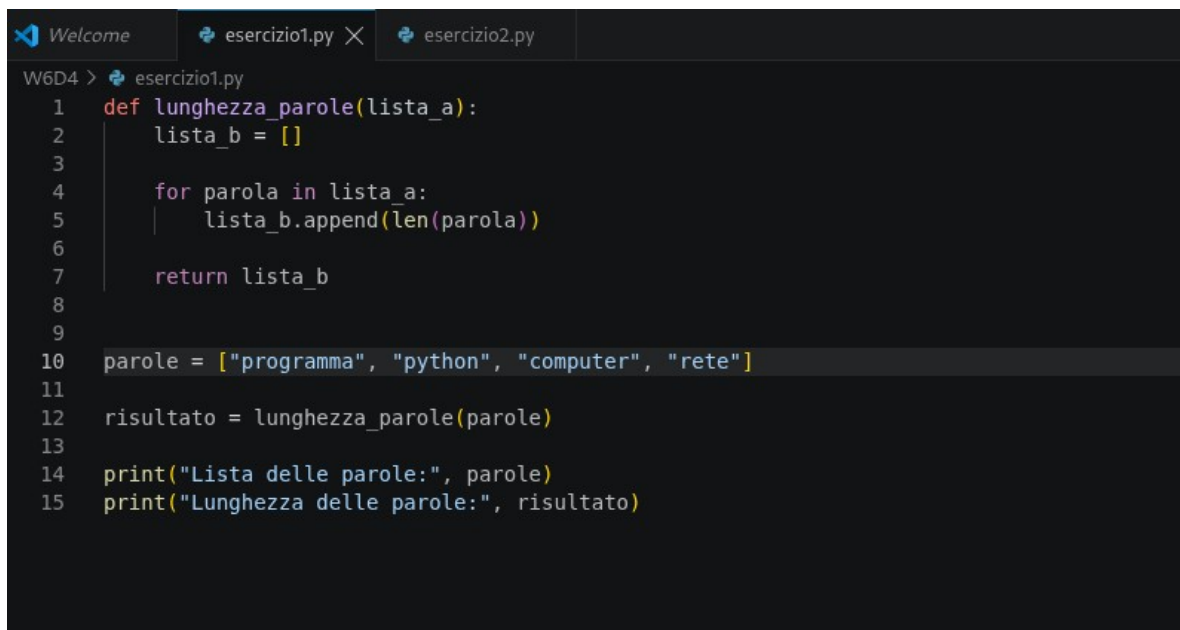
Per il primo esercizio ho creato un file Python chiamato: “esercizio1.py”.

La traccia richiedeva di creare una funzione che, partendo da una lista contenente alcune parole, restituisse una seconda lista formata da numeri interi.

Ogni numero deve rappresentare la lunghezza della parola corrispondente presente nella lista iniziale.

2.2 Codice dell’esercizio base

Figura 2 – Codice Python dell’esercizio base



```
W6D4 > esercizio1.py
1 def lunghezza_parole(lista_a):
2     lista_b = []
3
4     for parola in lista_a:
5         lista_b.append(len(parola))
6
7     return lista_b
8
9
10 parole = ["programma", "python", "computer", "rete"]
11
12 risultato = lunghezza_parole(parole)
13
14 print("Lista delle parole:", parole)
15 print("Lunghezza delle parole:", risultato)
```

2.3 Spiegazione del codice

2.3.1 Creazione della funzione

- **def lunghezza_parole(lista_a):**

Con def ho creato una funzione chiamata lunghezza_parole.

La funzione riceve come parametro lista_a, che rappresenta la lista contenente le parole da analizzare.

2.3.2 Creazione della seconda lista

- **lista_b = []**

All'interno della funzione ho creato una nuova lista inizialmente vuota.

Questa lista viene utilizzata per salvare la lunghezza di ogni parola presente nella lista iniziale.

2.3.3 Lettura delle parole

- **for parola in lista_a:**

Con il ciclo for il programma legge una parola alla volta presente nella lista.

Ad ogni passaggio del ciclo, la variabile parola contiene uno degli elementi della lista.

2.3.4 Calcolo della lunghezza

- **lista_b.append(len(parola))**

Per conoscere la lunghezza di ogni parola ho utilizzato la funzione len().

Ad esempio:

- programma = 9 caratteri
- python = 6 caratteri
- computer = 8 caratteri
- rete = 4 caratteri

Il metodo append() serve invece per aggiungere il risultato ottenuto alla lista lista_b.

In questo modo, ad ogni passaggio del ciclo, viene aggiunta la lunghezza della parola analizzata.

2.3.5 Restituzione del risultato

- **return lista_b**

Terminato il ciclo, la funzione restituisce la nuova lista contenente tutte le lunghezze calcolate.

2.3.6 Creazione della lista delle parole

- **parole = ["programma", "python", "computer", "rete"]**

Per verificare il funzionamento del programma ho creato una lista contenente quattro parole.

Ho scelto le parole programma, python, computer e rete, che hanno lunghezze differenti in modo da poter controllare facilmente il risultato.

2.3.7 Chiamata della funzione

- **risultato = lunghezza_parole(parole)**

Con questa istruzione viene richiamata la funzione e viene passata come parametro la lista parole.

Il valore restituito dalla funzione viene salvato nella variabile risultato.

2.3.8 Visualizzazione del risultato

- **print("Lista delle parole:", parole)**
- **print("Lunghezza delle parole:", risultato)**

Con le due istruzioni print() vengono visualizzate sia la lista iniziale sia la lista contenente la lunghezza delle parole.

2.4 Esecuzione e verifica

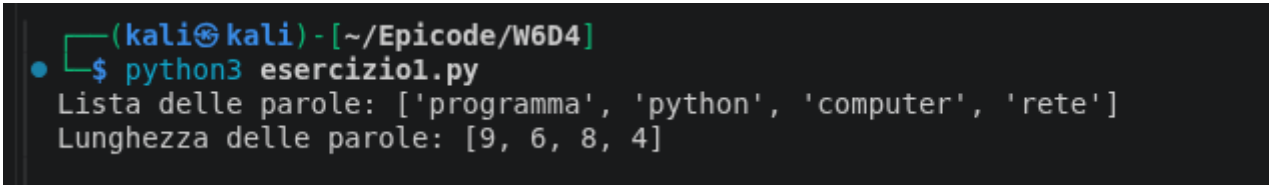
Per eseguire il programma ho utilizzato il terminale integrato di Visual Studio Code con il comando: “python3 esercizio1.py”.

Il risultato ottenuto è stato:

- **Lista delle parole: ['programma', 'python', 'computer', 'rete']**
- **Lunghezza delle parole: [9, 6, 8, 4]**

Il risultato è corretto perché ogni numero corrisponde alla lunghezza della rispettiva parola.

Figura 3 – Risultato dell’esecuzione dell’esercizio base



```
(kali@kali)-[~/Epicode/W6D4]
$ python3 esercizio1.py
Lista delle parole: ['programma', 'python', 'computer', 'rete']
Lunghezza delle parole: [9, 6, 8, 4]
```

3. Esercizio facoltativo

3.1 Creazione del file

Per il secondo esercizio ho creato un nuovo file Python chiamato: “esercizio2.py”.

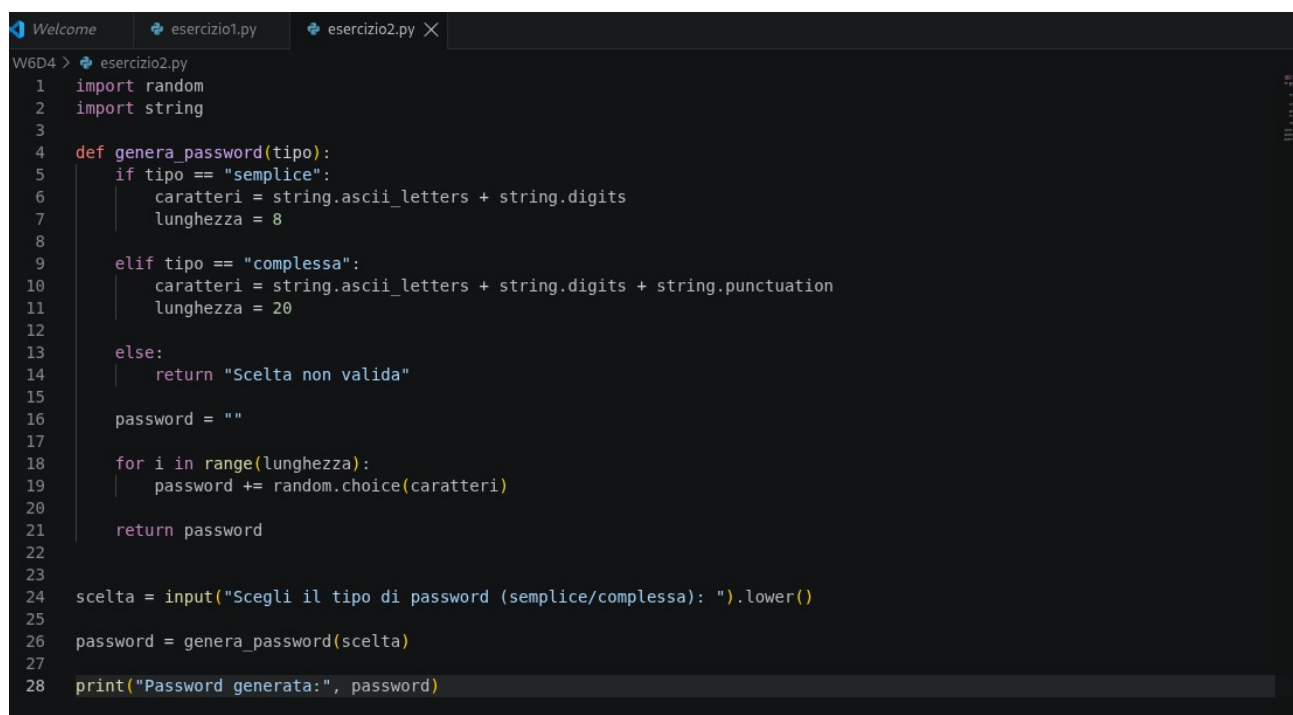
La traccia richiedeva di realizzare una funzione generatrice di password.

Il programma deve permettere all’utente di scegliere tra:

- una password semplice, composta da 8 caratteri alfanumerici;
- una password complessa, composta da 20 caratteri ASCII.

3.2 Codice dell’esercizio facoltativo

Figura 4 – Codice Python dell’esercizio facoltativo



```
W6D4 > esercizio2.py
1 import random
2 import string
3
4 def genera_password(tipo):
5     if tipo == "semplice":
6         caratteri = string.ascii_letters + string.digits
7         lunghezza = 8
8
9     elif tipo == "complessa":
10        caratteri = string.ascii_letters + string.digits + string.punctuation
11        lunghezza = 20
12
13    else:
14        return "Scelta non valida"
15
16    password = ""
17
18    for i in range(lunghezza):
19        password += random.choice(caratteri)
20
21    return password
22
23
24 scelta = input("Scegli il tipo di password (semplice/complessa): ").lower()
25
26 password = genera_password(scelta)
27
28 print("Password generata:", password)
```

3.3 Funzionamento del programma

Il programma permette all’utente di scegliere quale tipo di password generare.

Se viene scelta la modalità semplice, il programma genera una password di 8 caratteri composta da lettere e numeri.

Se invece viene scelta la modalità complessa, la password viene generata utilizzando lettere, numeri e caratteri speciali, per una lunghezza totale di 20 caratteri.

La password cambia ad ogni esecuzione perché i caratteri vengono scelti casualmente.

3.4 Spiegazione delle parti principali

3.4.1 Importazione dei moduli

- `import random`
- `import string`

All'inizio del programma ho importato i moduli `random` e `string`.

Il modulo `random` viene utilizzato per scegliere casualmente i caratteri della password.

Il modulo `string` mette invece a disposizione gruppi di caratteri già pronti, come lettere, numeri e simboli.

Non è stato necessario installare moduli aggiuntivi perché entrambi fanno parte della libreria standard di Python.

3.4.2 Creazione della funzione

- `def genera_password(tipo):`

Con `def` ho creato la funzione `genera_password`.

Il parametro `tipo` contiene la scelta effettuata dall'utente e permette al programma di capire quale password deve essere generata.

3.4.3 Password semplice

- `if tipo == "semplice":`
`caratteri = string.ascii_letters + string.digits`
`lunghezza = 8`

Se l'utente sceglie `semplice`, il programma utilizza lettere e numeri.

`string.ascii_letters` contiene le lettere maiuscole e minuscole, mentre `string.digits` contiene i numeri da 0 a 9.

La lunghezza viene impostata a 8 caratteri.

3.4.4 Password complessa

- `elif tipo == "complessa":`
`caratteri = string.ascii_letters + string.digits + string.punctuation`
`lunghezza = 20`

Se viene scelta la password complessa, oltre alle lettere e ai numeri vengono aggiunti anche i caratteri speciali tramite `string.punctuation`.

In questo caso la lunghezza viene impostata a 20 caratteri.

3.4.5 Controllo della scelta

- **else:**
return "Scelta non valida"

Se l'utente inserisce un valore diverso da "semplice" o "complessa", la funzione restituisce il messaggio Scelta non valida.

3.4.6 Creazione della password

- **password = ""**

La variabile password viene inizialmente creata come una stringa vuota.

I caratteri scelti casualmente verranno aggiunti uno alla volta al suo interno.

3.4.7 Generazione casuale dei caratteri

- **for i in range(lunghezza):**
password += random.choice(caratteri)

Il ciclo for viene ripetuto in base alla lunghezza scelta.

Ad ogni passaggio random.choice() seleziona casualmente un carattere dall'insieme disponibile e lo aggiunge alla password.

Se la password è semplice il ciclo viene eseguito 8 volte, mentre per quella complessa viene eseguito 20 volte.

3.4.8 Inserimento della scelta

- **scelta = input("Scegli il tipo di password (semplice/complessa): ").lower()**

Il comando input() permette all'utente di scegliere il tipo di password direttamente dal terminale.

Ho utilizzato anche .lower() per trasformare il testo inserito in minuscolo.

In questo modo, ad esempio, anche scrivendo SEMPLICE o Semplice, il valore viene trasformato in semplice.

3.4.9 Chiamata della funzione

- **password = genera_password(scelta)**

La scelta dell'utente viene passata alla funzione genera_password().

La password restituita viene poi salvata nella variabile password.

3.4.10 Visualizzazione della password

- `print("Password generata:", password)`

Infine il programma mostra nel terminale la password generata.

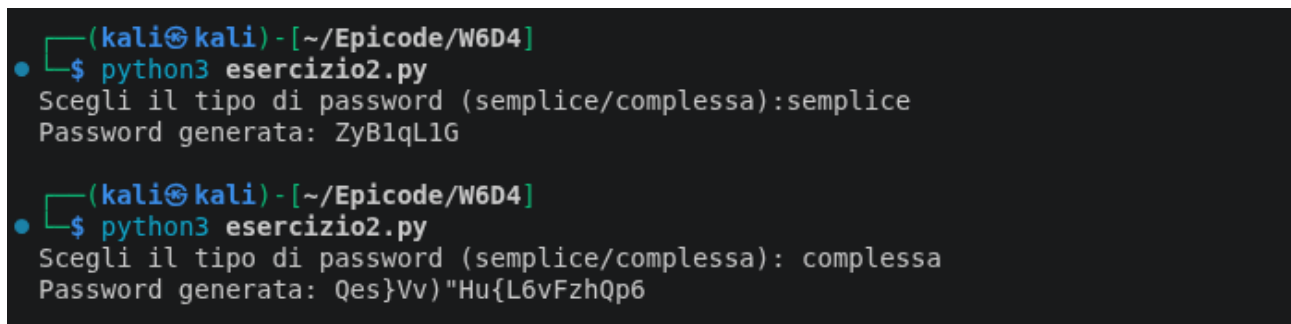
3.5 Verifica del funzionamento

Per verificare il programma ho effettuato due prove.

Nella prima esecuzione ho scelto “semplice” e il programma ha generato una password alfanumerica di 8 caratteri.

Successivamente ho eseguito nuovamente il programma scegliendo “complessa”. In questo caso è stata generata una password di 20 caratteri contenente anche simboli speciali.

Figura 5 – Generazione della password semplice e della password complessa



```
(kali㉿kali) - [~/Epicode/W6D4]
• $ python3 esercizio2.py
Scegli il tipo di password (semplice/complessa): semplice
Password generata: ZyBlqLlG

(kali㉿kali) - [~/Epicode/W6D4]
• $ python3 esercizio2.py
Scegli il tipo di password (semplice/complessa): complessa
Password generata: Qes}Vv)'Hu{L6vFzhQp6
```

4. Conclusioni

Nel complesso sono riuscito a completare entrambi gli esercizi e ad eseguirli correttamente utilizzando Visual Studio Code su Kali Linux.

Nel primo esercizio ho utilizzato una funzione, una lista e un ciclo for per leggere le parole e calcolarne la lunghezza tramite `len()`.

Nel secondo esercizio ho utilizzato anche i moduli `random` e `string` per generare password casuali. Questa parte mi è servita anche per capire meglio come utilizzare caratteri diversi all’interno di una stringa e come modificare il comportamento del programma in base alla scelta dell’utente.

Rispetto alle esercitazioni precedenti ho utilizzato Visual Studio Code invece dell’editor Nano e ho trovato più comodo poter scrivere il codice ed eseguirlo direttamente dal terminale integrato.